

RAI Robotics Competition Report

Semester1 Final Project



Submit to Dr. Poom Konghuayrob

Report By:

Hsu Myat Noe , Sec1

Moe Chan Myae Maung ,Sec2

Poonnachote Kaewsuwan , Sec3

Auto Robot

Part List

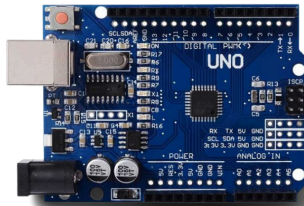
2x Yellow DC Motor



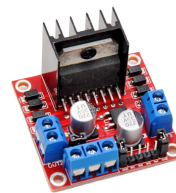
3x Ultra-distance sensors



1x Arduino UNO



1 x L298N Motor Drive Module

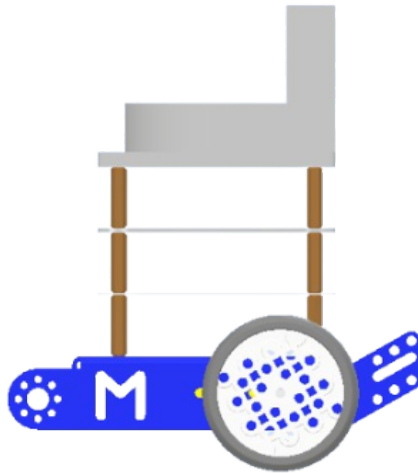
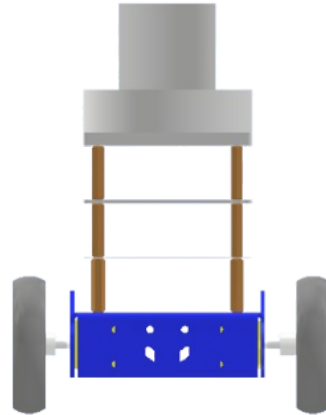
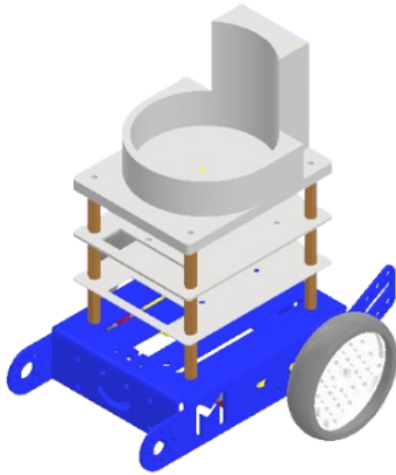


1x battery 7.4V



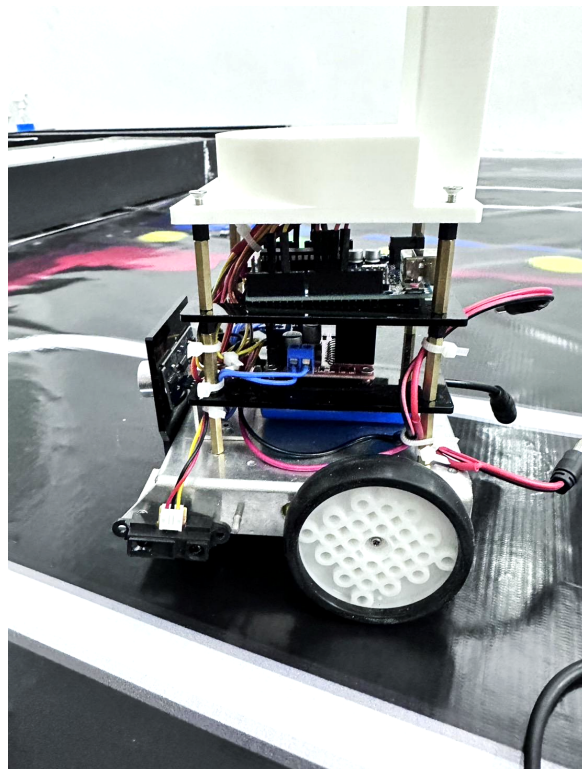
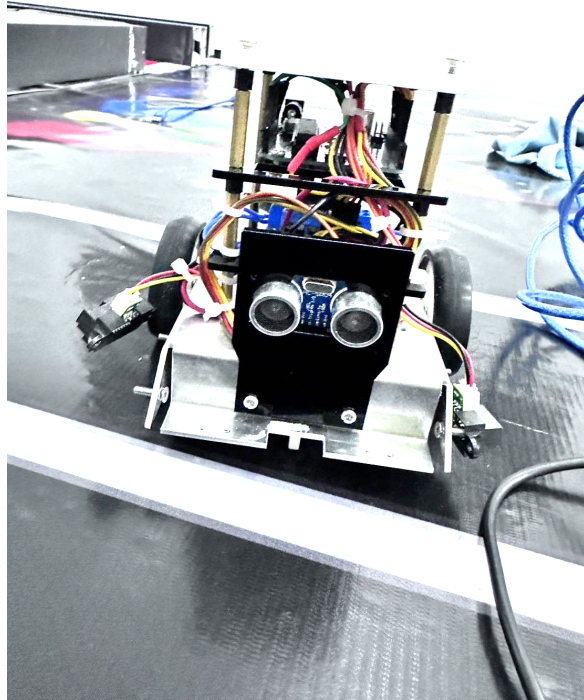
Assembly

Size: (W) 150 mm.x (L) 130 mm.x (H) 205 mm.

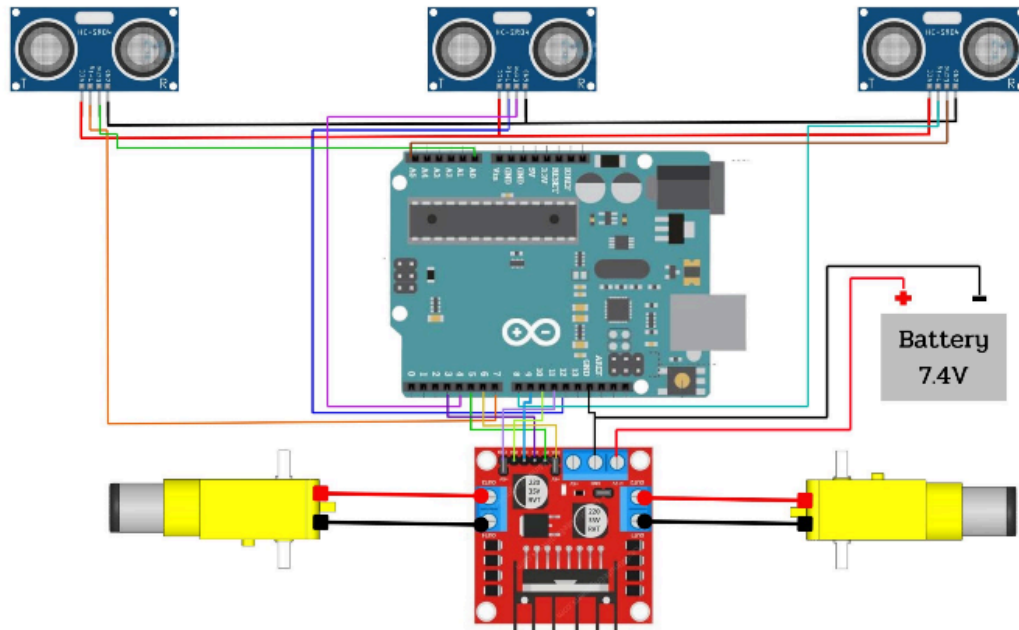


Mechanic parts of how we contain the ball. The first layer is for the motor driver and the second is for the UNO board.

Robot Photo



Electrical Diagram



Robot Video

 SuperCoolVideo.mp4

```

graph TD
    Start([Start]) --> Init[struct UltrasonicSensor, Motor, PID]
    Init --> Decl[Declare UltrasonicSensor sensors[3], Motor motors[2], PID pid]
    Decl --> Const[const int setpoint = 7;  
int baseSpeed = 150;  
const int frontDistanceThreshold = 12;  
const int frontDistanceThresholdSecondDetection = 15;  
int leftMotorSpeed = 0;  
int rightMotorSpeed = 0;  
int speedOffset = 0;  
int turn_time = 320;  
int forward_time = 600;  
int frontLeftWallDetectedCount = 0;  
int rightWallDetectedCount = 0;]
    Const --> Setup[Setup the pins]
    Setup --> FrontWall{if isFrontWallDetected()}
    FrontWall -- Yes --> StopFront[stopRobot();  
delay(500);]
    StopFront --> RightWall{if (isRightWallDetected()) && !isLeftWallDetected()}
    RightWall -- Yes --> TurnLeft[turnLeft();  
stopRobot();]
    RightWall -- No --> LeftWall{else if (isLeftWallDetected()) && isRightWallDetected()}
    LeftWall -- Yes --> StopLeft[stopRobot();]
    LeftWall -- No --> BothWall{else if (isLeftWallDetected()) && !isRightWallDetected()}
    BothWall -- Yes --> TurnRight[turnRight();  
stopRobot();]
    BothWall -- No --> Forward{else if (isLeftWallDetected()) && isRightWallDetected()}
    Forward -- Yes --> FollowWallPID1[followWallPID();]
    FollowWallPID1 --> A([A])
    Forward -- No --> ForwardNoWall{else if (isLeftWallDetected())}
    ForwardNoWall -- Yes --> FollowWallPID2[followWallPID();]
    FollowWallPID2 --> A
    ForwardNoWall -- No --> ForwardNoWallStop[Forward();  
stopRobot();]
    ForwardNoWallStop --> A
    B([B]) --> GetDist[getUltrasonicDistance(UltrasonicSensor* sensor)]
    GetDist --> Duration[duration * 0.034 / 2;]
    Duration --> End([END])
    FollowWallPID[followWallPID()] --> CalcPID[int leftDistance = getUltrasonicDistance(&sensors[2]);  
int error = leftDistance - setpoint;  
unsigned long currentTime = millis();  
double deltaTime = (currentTime - pid.lastTime) / 1000.0;]
    CalcPID --> DeltaTimeCheck{if (deltaTime > 0)}
    DeltaTimeCheck --> SetSpeed[setMotorSpeed(leftMotorSpeed,  
rightMotorSpeed);]
    SetSpeed --> FollowWallPID

```

Coding and explanation

```
struct UltrasonicSensor {  
    int trigPin;  
    int echoPin;  
};
```

```
struct Motor {  
    int forwardPin;  
    int backwardPin;  
    int speedPin;  
};
```

```
const double Kp = 8.0, Ki = 0.00001, Kd = 5.0;  
const int setpoint = 7, baseSpeed = 125, turn_time = 320, forward_time = 600;
```

```
UltrasonicSensor sensors[] = {  
    {8, A5}, // Right  
    {12, 4}, // Front  
    {7, A0} // Left  
};
```

```
Motor motors[] = {  
    {5, 3, 6}, // Motor 1  
    {9, 10, 11} // Motor 2  
};
```

```
double prevError = 0, integral = 0;  
unsigned long lastTime = 0;  
int speedOffset = 0, leftMotorSpeed = 0, rightMotorSpeed = 0;
```

```
void setup() {  
    Serial.begin(9600);  
    Serial.println(F("Initialize System"));
```

```
    for (UltrasonicSensor &sensor : sensors) {  
        pinMode(sensor.trigPin, OUTPUT);  
        pinMode(sensor.echoPin, INPUT);  
    }  
    for (Motor &motor : motors) {  
        pinMode(motor.forwardPin, OUTPUT);  
        pinMode(motor.backwardPin, OUTPUT);  
        pinMode(motor.speedPin, OUTPUT);
```

```
}  
}
```

```
void loop() {  
  if (isWallDetected(sensors[1], 9)) {  
    stopRobot();  
    delay(500);  
    if (isWallDetected(sensors[2], 18) && !isWallDetected(sensors[0], 18)) {  
      Serial.println("Front and Left wall detected ----- Turn Right!!!");  
      turnRight();  
      delay(350);  
    } else if (isWallDetected(sensors[0], 18) && !isWallDetected(sensors[2], 18)) {  
      Serial.println("Front and Right wall detected ----- Turn Left!!!");  
      turnLeft();  
      delay(turn_time);  
    } else if (isWallDetected(sensors[2], 18) && isWallDetected(sensors[0], 18)) {  
      Serial.println("All walls detected --- Stop");  
      stopRobot();  
    } else {  
      Serial.println("Front only detected, Turn Right");  
      turnRight();  
      delay(turn_time);  
    }  
    stopRobot();  
    delay(500);  
  } else if (isWallDetected(sensors[2], 18) && isWallDetected(sensors[0], 18)) {  
    Serial.println("Left and Right walls detected, following left wall");  
    followWallPID();  
  } else if (isWallDetected(sensors[2], 18)) {  
    Serial.println("Left wall detected, turning right");  
    turnRight();  
    delay(turn_time);  
    stopRobot();  
    delay(500);  
  } else if (isWallDetected(sensors[0], 18)) {  
    Serial.println("Right wall detected, moving forward and turning left");  
    moveForward();  
    delay(forward_time);  
    stopRobot();  
    turnLeft();  
    delay(turn_time);  
    stopRobot();  
    delay(500);  
  } else {
```

```

        Serial.println("No walls detected, moving forward");
        moveForward();
        delay(forward_time);
        stopRobot();
        delay(500);
    }
}

int getDistance(UltrasonicSensor *sensor) {
    digitalWrite(sensor->trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(sensor->trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(sensor->trigPin, LOW);

    int duration = pulseIn(sensor->echoPin, HIGH);
    return duration * 0.034 / 2;
}

bool isWallDetected(UltrasonicSensor sensor, int threshold) {
    return getDistance(&sensor) < threshold;
}

void followWallPID() {
    int leftDistance = getDistance(&sensors[2]);
    int error = leftDistance - setpoint;

    unsigned long currentTime = millis();
    double deltaTime = (currentTime - lastTime) / 1000.0;

    if (deltaTime > 0) {
        integral += error * deltaTime;
        double derivative = (error - prevError) / deltaTime;
        double controlSignal = Kp * error + Ki * integral + Kd * derivative;
        prevError = error;
        lastTime = currentTime;

        speedOffset = constrain(controlSignal, -100, 100);
        leftMotorSpeed = baseSpeed - speedOffset;
        rightMotorSpeed = baseSpeed + speedOffset;

        setMotorSpeed(leftMotorSpeed, rightMotorSpeed);
    }
}

```

```
void setMotorSpeed(int leftSpeed, int rightSpeed) {  
    digitalWrite(motors[0].forwardPin, HIGH);  
    digitalWrite(motors[1].forwardPin, HIGH);  
    digitalWrite(motors[0].backwardPin, LOW);  
    digitalWrite(motors[1].backwardPin, LOW);  
    analogWrite(motors[0].speedPin, rightSpeed);  
    analogWrite(motors[1].speedPin, leftSpeed);  
}
```

```
void stopRobot() {  
    for (Motor &motor : motors) {  
        digitalWrite(motor.forwardPin, LOW);  
        digitalWrite(motor.backwardPin, LOW);  
        analogWrite(motor.speedPin, 0);  
    }  
}
```

```
void moveForward() {  
    setMotorSpeed(200, 200);  
}
```

```
void turnLeft() {  
    digitalWrite(motors[0].forwardPin, HIGH);  
    digitalWrite(motors[1].forwardPin, LOW);  
    digitalWrite(motors[0].backwardPin, LOW);  
    digitalWrite(motors[1].backwardPin, HIGH);  
    analogWrite(motors[0].speedPin, 200);  
    analogWrite(motors[1].speedPin, 200);  
}
```

```
void turnRight() {  
    digitalWrite(motors[0].forwardPin, LOW);  
    digitalWrite(motors[1].forwardPin, HIGH);  
    digitalWrite(motors[0].backwardPin, HIGH);  
    digitalWrite(motors[1].backwardPin, LOW);  
    analogWrite(motors[0].speedPin, 200);  
    analogWrite(motors[1].speedPin, 200);  
}
```

Code Explanation

This code is for a robot that uses ultrasonic sensors and a Proportional-Integral-Derivative (PID) control system to navigate and avoid obstacles. The robot is equipped with sensors to detect walls on its left, right, and front sides, and it has motors controlled by a PID loop to adjust its path. Here's a breakdown of the main components and logic:

1. **Sensor and Motor Pin Setup:** The code defines pins for three ultrasonic sensors (left, right, and front) and for motor control. It sets the motor pin modes to control the robot's movement, including moving forward, turning left or right, and stopping.
2. **PID Control Constants:** The code defines constants (`Kp`, `Ki`, and `Kd`) for the PID controller, which are adjusted to keep the robot at a target distance (`setpoint`) from the wall. `Kp` is the proportional gain, which directly affects how aggressively the robot adjusts its direction to maintain a distance from the wall. `Ki` is the integral gain, a small value to account for minor adjustments, and `Kd` is the derivative gain, which smooths corrections to avoid oscillation.
3. **Wall Detection Functions:** Each ultrasonic sensor has a function (`isFrontWallDetected`, `isLeftWallDetected`, and `isRightWallDetected`) that calculates the distance to a wall. These functions return a Boolean value, indicating if the wall is within a predefined range, allowing the robot to detect obstacles on each side.
4. **Wall Following Logic:** Inside the `loop()` function, the robot checks each sensor's readings. Based on the presence of walls, the robot decides how to move:
 - If a wall is detected in front, the robot stops and chooses to turn left or right based on the readings from the side sensors.
 - If only a left or right wall is detected, the robot turns accordingly.
 - If no walls are detected, the robot moves forward.
 - If both side walls are detected, it activates `followWallPID()` to maintain a set distance from the left wall.
5. **PID-Based Wall Following:** The `followWallPID()` function calculates the error between the current distance to the left wall and the `setpoint`. Using this error, the function calculates the control signal based on the PID formula, adjusting the robot's motor speeds (`leftMotorSpeed` and `rightMotorSpeed`) to stay close to the wall. The PID controller ensures smoother and more stable adjustments, particularly in environments where maintaining a fixed distance from walls is essential.
6. **Robot Movements:** Helper functions (`stopRobot`, `Forward`, `turnLeft`, and `turnRight`) control specific movements. They set the motor directions and speed values to execute each movement, ensuring the robot can respond to obstacles effectively.

7. Timing Adjustments: Delays in each function (like `delay(turn_time)`) ensure the robot has time to complete turns or stops as it navigates through different scenarios.

The PID controller helps the robot maintain an optimal distance from obstacles and navigate effectively. This combination of sensors, control logic, and PID adjustments allows the robot to operate autonomously, respond to its surroundings, and avoid collisions.

Manual Robot

Part List

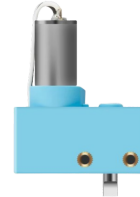
2x MS - 1.5A Servos



1x MG996R Servo



1x 180 Optical Encoder Motor



1x CyberPi



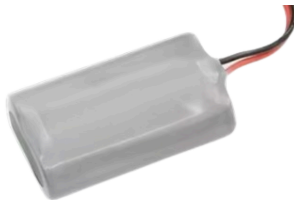
1x Mbot2 mbot neo shield



2x Yellow DC motors

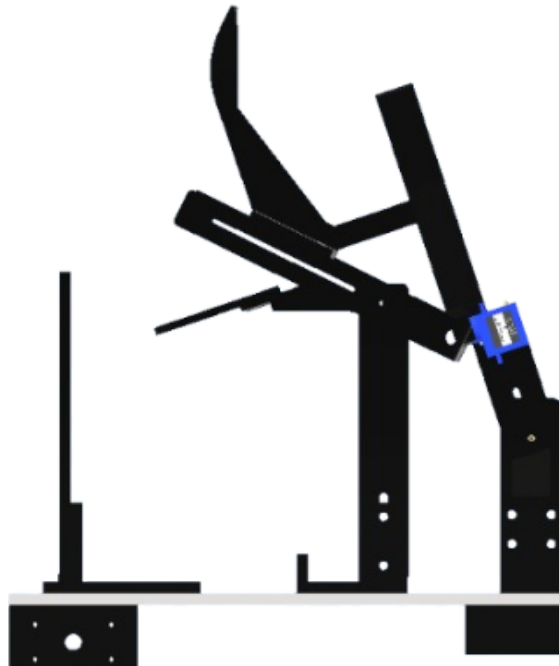
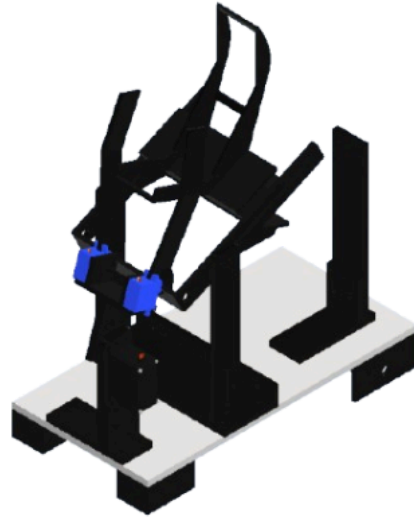
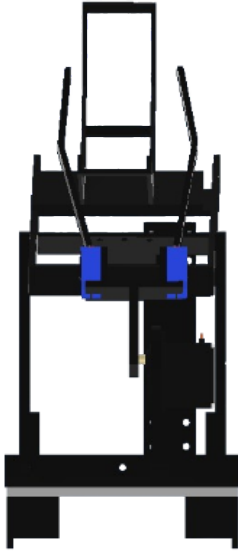


1x battery 7.4V

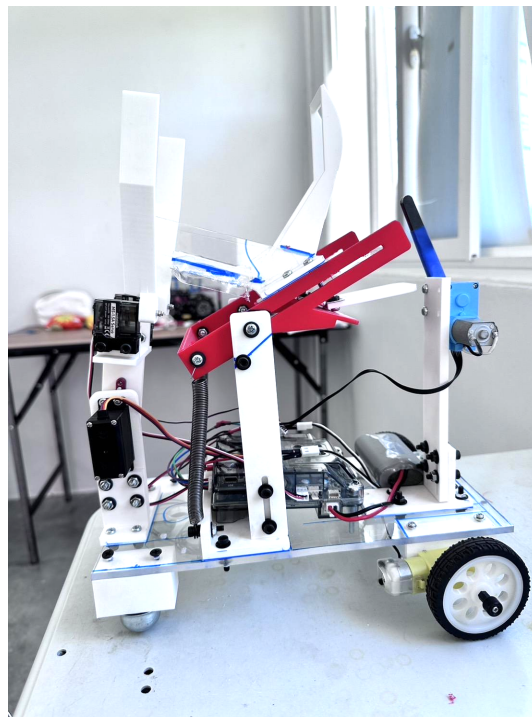
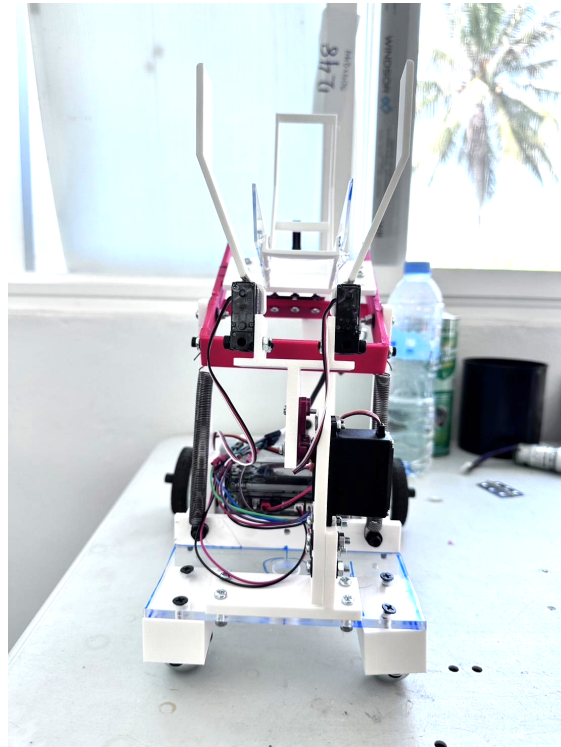


Assembly

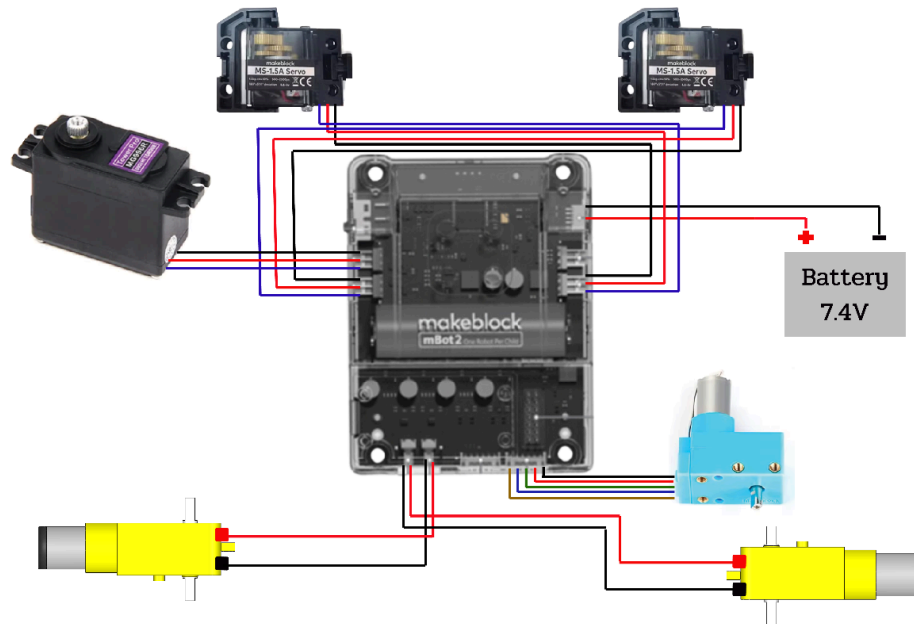
Size: (W) 142 mm.x (L) 282 mm.x (H) 350 mm.



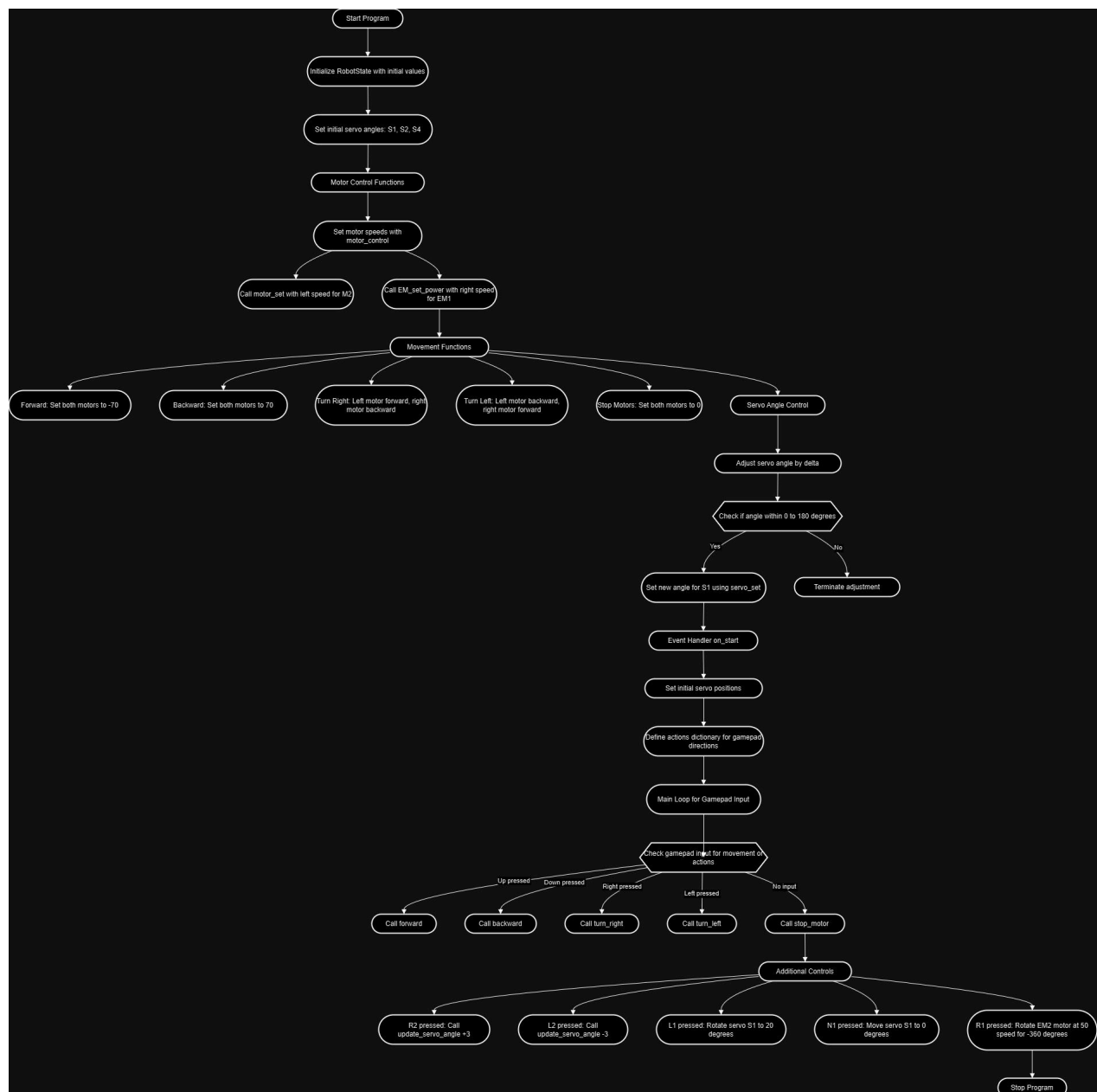
Robot Photo



Electrical Diagram



Flow Chart



Code and explanation

```
import mbot2, event, time, cyberpi, gamepad

class RobotState:
    def __init__(self):
        self.servo_angle = 130
        self.motor_speeds = {'M2': 0, 'EM1': 0}
        self.servo_angles = {'S1': 130, 'S2': 0, 'S4': 90}

robot_state = RobotState()

def motor_control(left_speed, right_speed):
    mbot2.motor_set(left_speed, "M2")
    mbot2.EM_set_power(right_speed, "EM1")

def forward():
    motor_control(-70, -70)

def backward():
    motor_control(70, 70)

def turn_right():
    motor_control(70, -70)

def turn_left():
    motor_control(-70, 70)

def stop_motor():
    motor_control(0, 0)

def update_servo_angle(delta):
    if 0 <= robot_state.servo_angle + delta <= 180:
        robot_state.servo_angle += delta
        mbot2.servo_set(robot_state.servo_angle, "S1")
        time.sleep(0.02)
```

@event.start

def on_start():

 mbot2.servo_set(robot_state.servo_angles["S1"], "S1")

 mbot2.servo_set(robot_state.servo_angles["S4"], "S4")

 mbot2.servo_set(robot_state.servo_angles["S2"], "S2")

actions = {

 'Up': forward,

 'Down': backward,

 'Right': turn_right,

 'Left': turn_left

}

while True:

 moving = False

 for key, action in actions.items():

 if gamepad.is_key_pressed(key):

 action()

 moving = True

 if not moving:

 stop_motor()

 if gamepad.is_key_pressed('R2'):

 update_servo_angle(3)

 elif gamepad.is_key_pressed('L2'):

 update_servo_angle(-3)

 if gamepad.is_key_pressed('L1'):

 mbot2.servo_drive(mbot2.servo_get("S1"), 20, 0, 70)

 if gamepad.is_key_pressed('N1'):

 mbot2.servo_drive(mbot2.servo_get("S1"), 0, 0, 90)

 if gamepad.is_key_pressed('R1'):

`mbot2.EM_turn(-360, 50, "EM2")`

1. Class RobotState

This class specifies a fundamental framework for storing the robot's current state, including motor speeds and servo angles.

The initial value of `servo_angle` is 130 degrees.

`motor_speeds` Motor speed values are stored in a dictionary for ports "M2" and "EM1."

The initial angles for servos labeled "S1," "S2," and "S4" are stored in the `servo_angles` dictionary.

2. Functions of Motor Control

By adjusting motor speeds, these functions control the robot's motion.

The left and right motors' speeds are set by `motor_control(left_speed, right_speed)`.

`(left_speed, "M2") mbot2.motor_set` regulates the motor on "M2."

`(right_speed, "EM1") mbot2.EM_set_power` regulates the motor on "EM1."

Functions of Movement:

`forward()`: Sets both motors to -70 (probably in reverse) to move the robot forward.

`reverse()`: Uses both motors to move the robot backward at +70.

`turn_right()`: For a right turn, it sets the left motor forward and the right motor backward.

`turn_left()`: For a left turn, it sets the right motor forward and the left motor backward.

`stop_motor()`: Sets speeds to 0 to halt both motors.

3. Servo Angle Control

`update_servo_angle(delta)`: Modifies the servo "S1"'s angle by delta, which can be either +3 or -3 degrees.

updates the servo angle after making sure the angle remains between 0 and 180°.

4. The on_start event handler

Initialisation: This function sets the servos to their initial angles specified in `robot_state` at the beginning of the program.

Dictionary of actions: translates the Up, Down, Right, and Left gamepad directions into movement functions.

Main Loop: Constantly monitors gamepad inputs to regulate the robot's actions:

When a directional button (Up, Down, Right, or Left) is pressed, the robot moves in the appropriate direction.

The robot is stopped by `stop_motor()` if no movement button is pushed.

`Update_servo_angle()` is called by the R2 and L2 buttons to regulate servo angle modifications.

Extra Controls for the Gamepad:

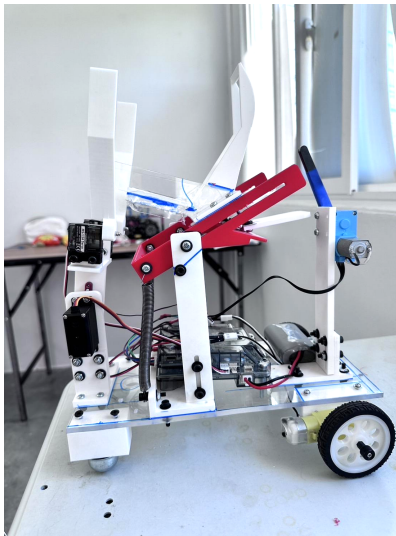
Servo "S1" is rotated by L1 to 20 degrees.

Servo "S1" is moved to 0 degrees by N1.

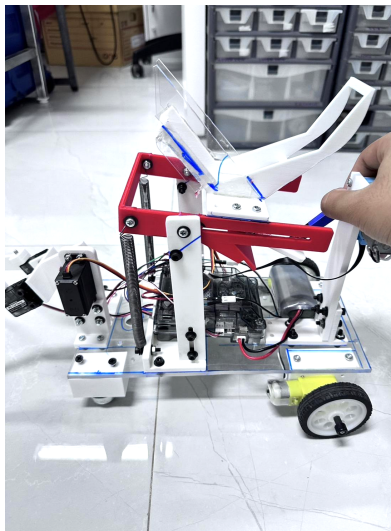
R1: Starts the motor on "EM2" to rotate at 50 rpm for a complete -360° revolution.

How does our shooter work?

Our shooter mechanism primarily uses springs to launch the ball, with two springs working together to generate a stronger force for returning to the original position. To amplify this force, we've incorporated an encoder motor that pushes the shooter in the opposite direction of the spring tension, building up even more resistive force. This setup allows the motor to compress the springs further, creating a higher potential energy that's released when the motor stops pushing. The encoder motor also enables precise control over the loading phase, making sure we reach the ideal force consistently.

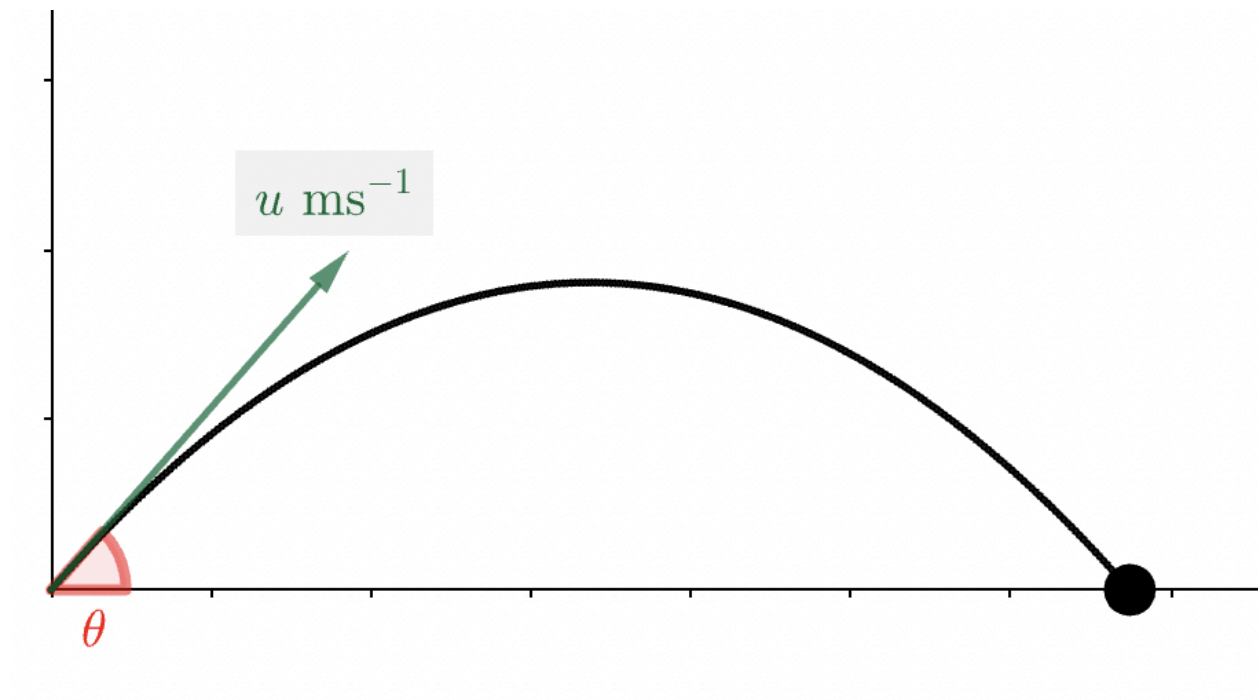


Before Extend



After Extend

Projectile



Potential Energy in the Spring (PE_{spring}): $PE_{\text{spring}} = \frac{1}{2}kx^2$

As the spring releases, the potential energy becomes kinetic energy: $KE = \frac{1}{2}mv^2$

Since energy is conserved, $\frac{1}{2}kx^2 = \frac{1}{2}mv^2$

Solving for Velocity: $v = \sqrt{\frac{kx^2}{m}}$

Horizontal Velocity = $v \cos(\theta)$

Vertical Velocity = $v \sin(\theta)$

Time of Flight(T) = $T = \frac{2v \sin(\theta)}{g}$

Range (R) = $R = v \cos(\theta) \times \frac{2v \sin(\theta)}{g}$ ←Horizontal Range